

Hoja de ruta del día

- **Módulo:** Machine Learning
- **Semana / Sesión:** Semana 16 - Fase 3: Evaluación y optimización
- **Temática central:** Diagnóstico del dilema sesgo-varianza y optimización de hiperparámetros con GridSearch
- **Stack requerido:** Python, Scikit-Learn, Pandas, NumPy, Matplotlib

Objetivos de aprendizaje del día

- **Objetivo 1:** Diagnosticar problemas de subajuste (alto sesgo) y sobreajuste (alta varianza) mediante el análisis del comportamiento de los errores de entrenamiento y validación al modificar la complejidad de un modelo.
- **Objetivo 2:** Implementar procesos de optimización de hiperparámetros mediante GridSearchCV utilizando la métrica estándar R^2 para localizar el punto de equilibrio óptimo.
- **Objetivo 3:** Construir un Pipeline robusto en Scikit-Learn que encapsule el escalado de características y el estimador, garantizando la ausencia de fugas de datos (data leakage) durante la validación cruzada.
- **Objetivo 4:** Evaluar cuantitativamente la estabilidad de un algoritmo midiendo la desviación estándar del rendimiento entre los diferentes pliegues (folds) de la validación cruzada junto con su coste computacional.

Bibliografía

- Documentación oficial 1: [Scikit-Learn API: GridSearchCV](#)
- Documentación oficial 2: [Scikit-Learn User Guide: Tuning the hyper-parameters of an estimator](#)
- Documentación oficial 3: [Scikit-Learn API: Pipeline](#)
- Documentación oficial 4: [Scikit-Learn User Guide: Validation curves: plotting scores to evaluate models](#)

Ejemplo de implementación práctica

El siguiente fragmento de código ilustra el patrón de diseño correcto para estructurar la búsqueda de hiperparámetros dentro de un flujo de trabajo aislado, utilizando una métrica de regresión estándar sin resolver la lógica del reto planteado:

```
Python
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

# Diseño de un flujo unificado para evitar la fuga de información
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('regressor', Ridge())
])

# Espacio de búsqueda para evaluar la penalización del modelo
param_grid = {
    'regressor__alpha': [0.1, 1.0, 10.0, 100.0]
}

# Configuración del optimizador empleando validación cruzada y R²
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1
)
# El ajuste (.fit) y el diagnóstico fino se delegan a la investigación autónoma
```

Reto práctico de código

Contexto del problema

Una entidad financiera de impacto social está desarrollando un modelo predictivo para evaluar el índice de riesgo de impago en créditos de vivienda. Disponen de variables socioeconómicas y demográficas de los solicitantes. El objetivo de negocio es predecir con exactitud dicho índice para agilizar las concesiones sin poner en peligro los fondos del programa.

Generación del conjunto de datos

Para trabajar de manera uniforme, inicia tu script ejecutando el siguiente bloque de código que genera el entorno de datos simulado:

```
Python
import numpy as np
import pandas as pd

def generar_dataset_riesgo():
    np.random.seed(42)
    n_muestras = 1000

    # Creación de variables: ingresos, código postal codificado y ruido
    ingresos = np.random.uniform(15000, 80000, n_muestras)
    codigo_postal_riesgo = np.random.choice([0, 1, 2], size=n_muestras, p=[0.2,
0.6, 0.2])

    # Relación no lineal subyacente con ruido irreducible inyectado
    riesgo_real = 0.000005 * (ingresos - 40000)**2 + 15 * codigo_postal_riesgo
    + np.random.normal(0, 10, n_muestras)

    df = pd.DataFrame({
        "ingresos": ingresos,
        "zona_cod": codigo_postal_riesgo
    })
    target = pd.Series(riesgo_real, name="indice_riesgo")
    return df, target

X_df, y_series = generar_dataset_riesgo()
```

Componente de incertidumbre

El conjunto de datos presenta ruido inherente y relaciones no lineales. Los modelos con restricciones severas sufren de subajuste (alto sesgo), mientras que los modelos excesivamente complejos se vuelven inestables entre particiones de datos (alta varianza). Debes estructurar un script que investigue y controle esta situación cumpliendo lo siguiente:

1. Encapsular el preprocesamiento y el algoritmo `DecisionTreeRegressor` dentro de un objeto `Pipeline`.
2. Configurar un espacio de búsqueda (`param_grid`) diseñado para explorar la complejidad del árbol. Debes hacer variar el parámetro `max_depth` (desde poca profundidad hasta un crecimiento libre) y el parámetro `min_samples_split`.
3. Realizar la optimización con `GridSearchCV` configurando explícitamente el criterio de evaluación con la métrica de determinación R^2 .
4. Controlar la ejecución del proceso mediante estructuras de manejo de excepciones. Al finalizar, debes extraer programáticamente de los atributos del `GridSearch`: la mejor configuración de hiperparámetros, el valor del mejor R^2 , el tiempo medio de cómputo empleado para entrenar dicho modelo y la desviación estándar de las puntuaciones del set de validación para diagnosticar el nivel de varianza.

Restricción ética y responsabilidad

El conjunto de datos incluye la variable `zona_cod` (código postal del solicitante), la cual está correlacionada de forma indirecta con áreas residenciales de bajos ingresos históricos. Al suavizar la varianza del modelo aplicando restricciones muy severas a la estructura del árbol, el algoritmo tiende a sobresimplificar las predicciones, asignando de manera automática un riesgo genéricamente elevado a los códigos postales históricamente vulnerables, lo que en la práctica se conoce como segregación financiera (*redlining*).

Analiza los resultados obtenidos y redacta una respuesta técnica concisa a la siguiente cuestión:

- ¿Es éticamente defendible aceptar un modelo con mayor varianza matemática (menor estabilidad entre particiones) si se demuestra empíricamente que reduce la discriminación algorítmica sistémica hacia colectivos desfavorecidos al evaluar los residuos por zona?